GOSIM

GOSIM Shenzhen 2026

October 16-17, 2026 Shenzhen, China



GOSIM

GOSIM Shenzhen 2026

UCM：从KV Cache加速走向智能体规模推理的全生命周期管理

王聪



CONTENTS

Part 01 AI Coding场景下的流量特征分析

Part 02 KV Cache推理加速系统设计

Part 03 UCM：面向LLM推理的统一KV Cache基础设施



01

AI Coding场景下的流量特征分析



AI Coding 已是一种新的推理负载

一次请求 · Request

8.8 步	LLM 调用 —— 一条指令，多步自主推进
10.8 次	工具调用 —— 执行 → 观察 → 再执行
61% : 38%	请求内时间：工具 vs 模型生成
4.3 min	端到端时长（p90 6.4 min）

一个会话 · Session

10.1 个	请求 —— 人会反复回来（p99 达 161）
92 次	工具调用累计
≈ 7 : 1	工具触发步： 用户触发步
7.5 h	平均跨度（p99 ≈ 7 天）

和 Chatbot 不一样 | AI Coding

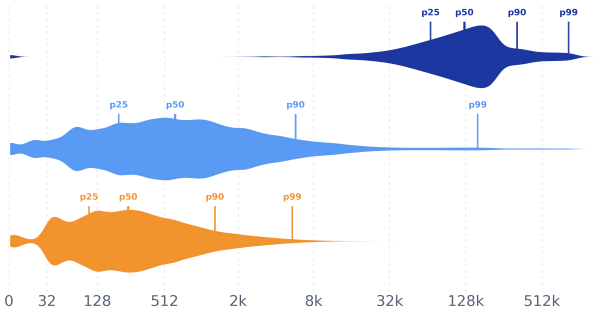
— 一问一答 · 答完即止
→ **一条指令 · 长链路自主执行**

— 上下文轻 · 一次装得下
→ **上下文重 · 边干边涨**

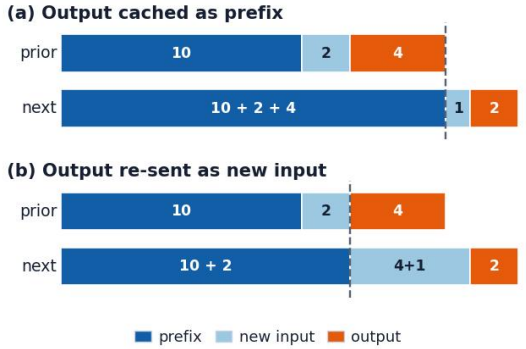
— KV 用完即丢
→ **KV 与会话同寿命 · 跨请求复用**

01	**输入形态**	读多写少
02	**缓存复用**	只增不减
03	**时间结构**	人机交互成主要瓶颈
04	**缓存失效**	保与丢两难

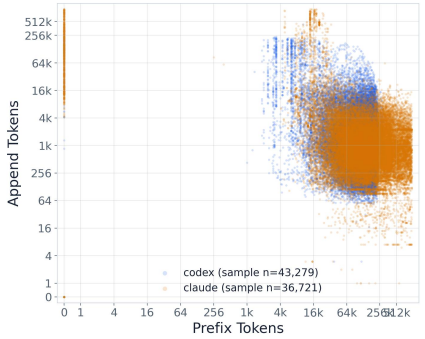
特征1：读多写少



同轴对比：三分布分层，读远多于写



机制示意：上一步输出的两种归宿（折进前缀 / 作为新增重发）



小前缀携大新增（冷启动），大前缀只带小新增

重放检验（相邻轮）	Claude	Codex
前缀连续性：下一步 ≈ 上一步输入	2 tok	350 tok
前缀误差 ±1% 内	94.1%	79.3%
新增包含上一步输出	97.3%	94.4%
每步新增 raw → 去重放	967 → 285	1,116 → 787

重放链路：下一步输入 ≈ 上一步全部输入 + 上一步输出

141K / 120K

中位前缀 token（Claude / Codex）

每一步都在重放的历史上下文

≈400×

中位输入 vs 中位输出（354 / 188）

算力与显存压力在 prefill 重放，不在 decode

172K vs 7.5K

平均每步总输入 vs 真正新增

输入体量由历史累积主导，而非当轮新内容

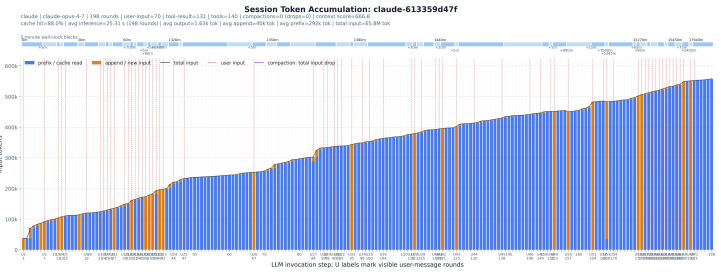
92 万

前缀 p99（Claude），逼近百万级上下文

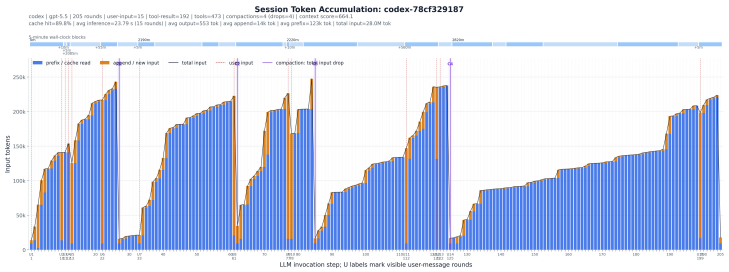
全负载累计输入 114.2B token

数据来源：TraceLab

特征2：极高的前缀命中率



Claude 会话：高蓝基座 + 薄橙顶，几乎不压缩



Codex 会话：压缩-再累积（C 标记）



按步骤数：近九成步骤命中 ≥ 0.9



按新增加权：token 成本集中在低命中步骤

95.6%

token 加权前缀命中率

工具触发步 97.8% ｜ 用户触发步 83.2%

98.3%

步骤上下文单调增长

仅 8.8% 会话压缩；单会话累计输入最高 9,200 万

16%

每步重算 token 中真正新鲜的部分

84% 本可由缓存提供，丢了按 6.3× 放大重算

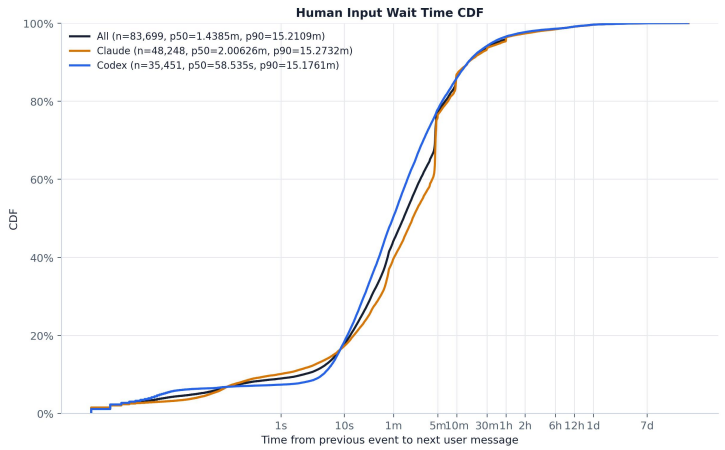
58.6%

成本来自缓存读

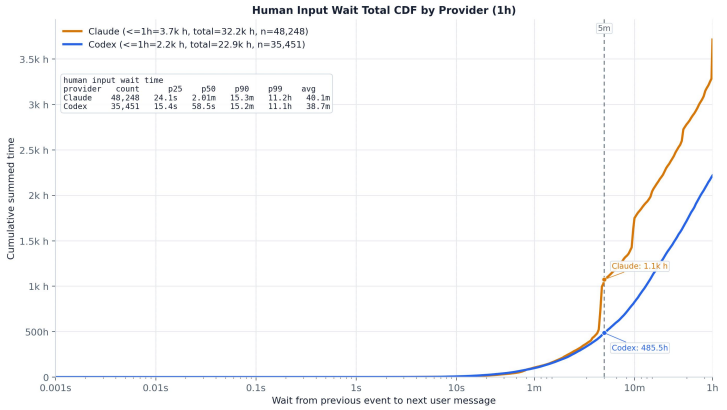
新输入 30.5% ｜ 输出仅 11% —— 重读上下文才是账单大头

数据来源：TraceLab

特征3：人机交互成为瓶颈



人类等待 CDF：主体秒级到分钟级



按时长加权：长尾贡献几乎全部空闲时间

会话墙钟时间去向



■ 人类思考 91.6%　■ 工具执行 5.1%　■ LLM 生成 3.2%

86s → 11h

人类等待 p50 → p99

等待横跨 7 个数量级

~90%

空闲时长来自 >1h 的等待

5min 内结束的等待仅占空闲时长的 2.8%

29×

人类空闲 ÷ 模型生成时间

GPU 不是瓶颈，人是最长的等待

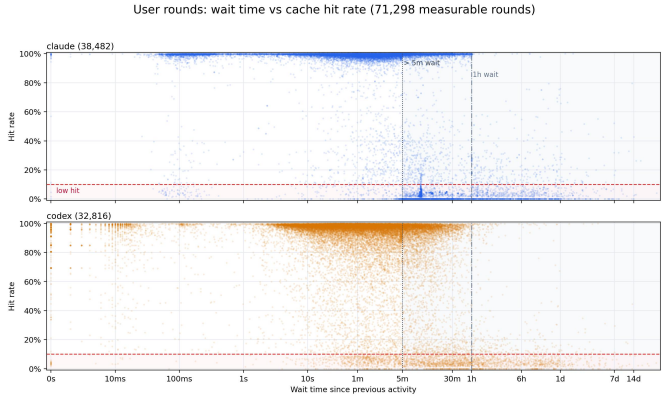
31 天

最长的一次人类等待

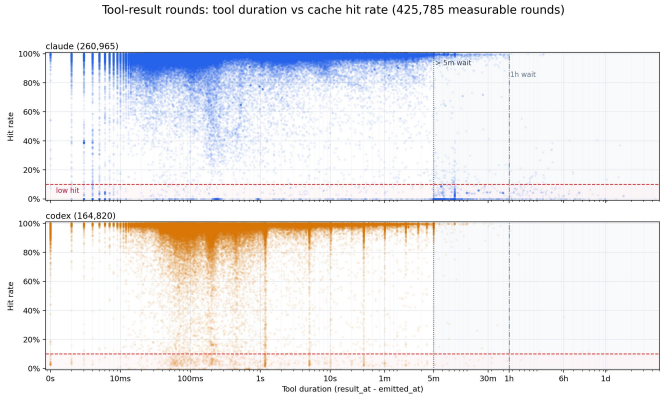
被搁置的会话仍会回来继续工作

数据来源：TraceLab

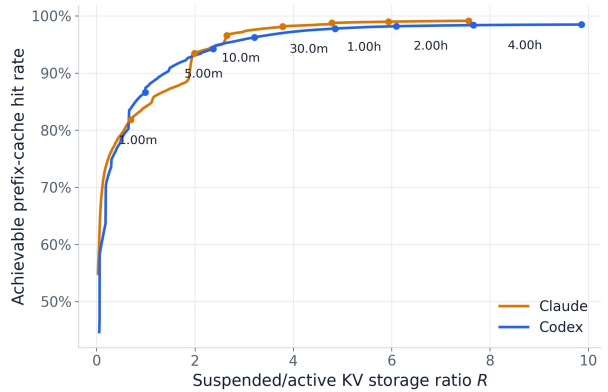
特征4：停顿即失效



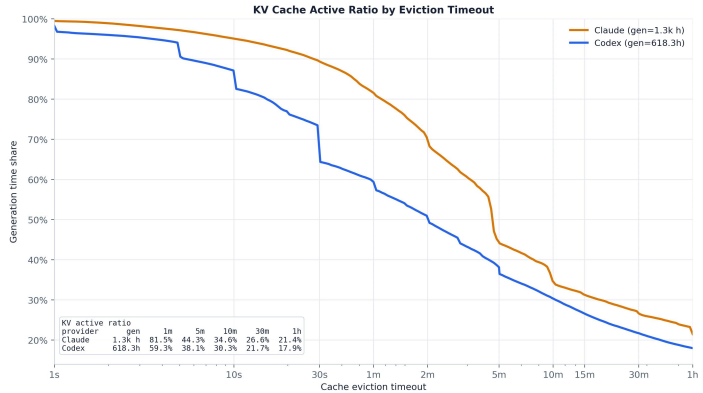
人类空闲 vs 命中率：>5min 尾巴变粗



工具等待 vs 命中率：几乎不受空闲影响



TTL 帕累托：命中 ↑ 则存储比 ↑，拐点 ≈5min



TTL=1h：KV 活跃复用占比仅 17.9%–21.4%

86%

低命中用户步等待 >5min

命中率 <10% 的 10,377 个用户步，失效可归因

93.7% @ 2.1×

TTL=5min：可达命中 / KV 存储比

现网部署点对应有效驱逐时间仅 ≈9min

98.6% @ 5.2×

TTL=1h：边际收益递减

命中率只多 5 个百分点，存储比再翻倍

-54% / -15.8%

思考期保缓存（反事实）：新增输入 / 成本

上界估计：假设全部以 cache-read 价命中

数据来源：TraceLab

总结

特征	趋势特征	对成本结构的含义
读多写少	输入由历史上下文重放主导，读远多于写	→ 不缓存，每步都在重算历史
前缀命中率极高	绝大多数输入可由前缀缓存命中，真正新增占比很小	→ 复用是常态，收益占大头
等待长尾效应	人机交互主导墙钟时间，会话被拉长且随时回来继续	→ KV 必须跨请求存活，与会话同寿命
停顿即失效	命中率随停顿快速衰减；保留越久，收益与存储代价同步上升	→ 持有有代价——保与丢两难

大语言模型推理进入规模化阶段后，KV Cache 已从推理过程的中间产物变为决定成本与体验的第一性资源。

02

KV Cache推理加速系统设计

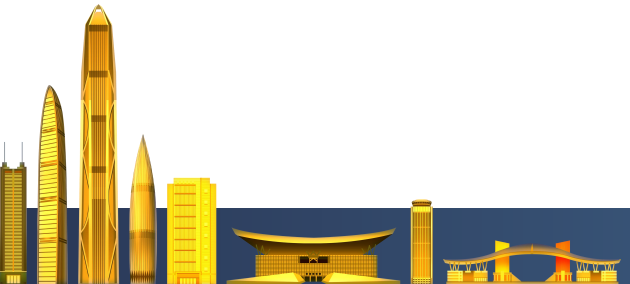


代价不会消失，只会转移

KV Cache 总成本 = 重算成本 + 驻留成本 + 搬运成本（+ 负载不均衡与运维成本）

重算：未命中时重新执行 Prefill | 驻留：保留时长 × 所在存储层级 | 搬运：跨层 / 跨节点的数据移动

优化动作	减少的成本	转移后的成本
不保留 KV	减少驻留与写入	增加重复 Prefill 计算
延长保留时间	减少缓存失效后的重算	增加容量占用与介质磨损
扩大热 KV 共享	减少重复计算，放宽亲和约束	增加网络与共享层管理开销
坚持本地亲和	减少 KV 搬移与远端读取	可能增加排队与算力闲置



四种候选架构的成本结构

热层是否共享（跨节点复用）× 容量是否独立扩展（算存解耦）——两个独立选择轴，组合出四种架构

架构	存储拓扑	减少的成本	转移后的成本
A · 本地为主	本地热缓存 + 本地 SSD	远端访问；共享系统建设	容量随计算绑定；重复存储 / 重算、亲和排队
B · 热层共享	共享热层 + 本地 SSD	跨节点热 KV 重算；部分亲和约束	共享层容量、网络与管理；冷数据仍绑定节点
C · 热层共享 + 容量解耦	共享热层 + 独立存储池	本地容量约束；部分热 KV 调度约束	两层资源投入与数据移动；元数据与运维
D · 容量解耦	本地热缓存 + 独立存储池	容量与计算绑定；长生命周期保留限制	远端回源与回迁；热 KV 亲和仍需权衡

C 同时引入热共享与容量解耦两类复杂度，收益需分别验证；C 并非 D 的必然升级，D 可以是长期选择；B 在已有本地 SSD 且热共享收益显著时成立

四种架构无绝对优劣，差异在于成本结构；实际性能取决于业务负载，需通过测量验证

亲和与共享：调度在本地命中与负载均衡之间权衡

调度决定时延与资源成本发生在哪里：留在亲和节点，或转向空闲节点

路径一 · 留在亲和节点

· **已有目标 KV，节点繁忙**

· → 本地加载：减少搬移与重算

成本：排队时延增加，其他节点算力闲置

路径二 · 转向空闲节点

· **算力可用，目标 KV 不在本地**

· → 远端加载，或重新执行 Prefill

成本：网络传输、远端读取或重算

比较：亲和节点的排队等待 + 本地加载 vs 空闲节点的等待 + 远端加载 / 重算

两者的相对成本取决于队列状态、热点分布与远端读取开销；共享扩大候选集合，但不自动实现负载均衡

分工：调度器决定请求放置；缓存层提供 KV 位置、可访问性与访问成本信息，不替代调度决策

优化目标是 **SLO 约束下的集群有效吞吐与单位 Token 成本**，而非 Prefix 命中率

部署条件决定初始选型

规模帮助形成初始判断，实际瓶颈决定最终选择

百卡左右 · 无专门存储团队

优先评估 A

- 系统建设与运维成本最敏感
- 以本地热缓存与本地 SSD 起步，保留重算兜底
- 允许偏离亲和，不为命中牺牲调度

→ 本地容量不足或已有成熟存储时，评估 D

数百至数千卡 · 多模型，扩缩容增多

优先比较 A / D

- 容量与计算扩展开始错配
- 节点扩缩容导致缓存反复冷启动
- 容量与生命周期被节点绑定 → 评估独立存储

→ 热共享收益明确时，再评估 B / C

数千至万卡 · 多租户，存算独立扩容

重点比较 C / D

- KV 工作集大、热点集中；PD 分离成熟
- KV 位置开始影响全局调度
- 亲和排队损失大 → 评估共享热层

→ 热点局部或远端读取足够快时，D 仍可能更经济

容量估算：KV 容量 ≈ 每 Token KV 大小 × KV 净产生速率 × 目标保留时间

再按复用去重、淘汰与冗余修正，并验证峰值回源带宽

规模不是架构升级的必然路线：由实测瓶颈驱动演进，每次只引入一种复杂度

架构选型之后：KV Cache 的四个共性问题

架构回答资源放在哪里；无论选择哪种拓扑，以下问题都存在，且随负载动态变化

复用 · 保留多久

哪些 KV 值得保留，保留多久，何时回收

· 由访问模式与业务负载决定，随时间变化
· 过期不回收挤占热层，误回收引发重算

→ 主导驻留成本

放置 · 保留在哪

KV 保留在哪里，如何与算力调度协作

· 亲和提升命中，但约束调度自由
· 跨节点共享扩大候选集合，不自动均衡负载

→ 影响重算成本与排队时延

迁移 · 如何移动

何时预取、何时搬移，走什么路径

· 用搬运成本换调度自由与容量利用率
· 传输效率决定远端数据是否真正可用

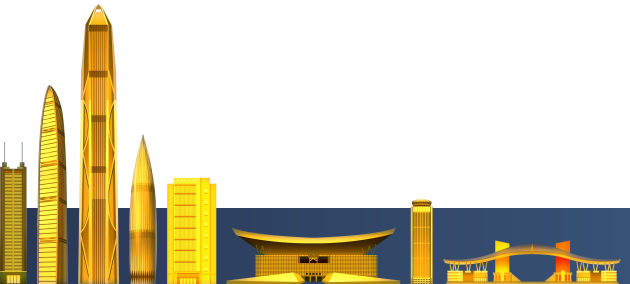
→ 主导搬运成本

兜底 · 压力去向

热层容量不足时，压力转向哪里

· 有副本 → 远端回源；无副本 → Prefill 重算
· 回源划算条件：C_read + C_transfer < C_recompute

→ 决定峰值尾时延与兜底能力



03

UCM：面向LLM推理的统一KV Cache基础设施



UCM：面向 LLM 推理的统一 KV Cache 管理层

① 应用与推理引擎 Applications & Engines

产生并消费 KV Cache

多模态上下文
图像 · 视频 · 文本

会话上下文
文本

▼ Engine Adapters · 引擎适配器（可插拔）

② 统一缓存 Unified Cache KV Cache Lifecycle

管理缓存的使用生命周期

复用 Reuse
匹配 · 检索 · 加载
前缀与 Chunk 复用

放置 Placement
分层 · 卸载 · 驱逐
跨层级保留管理

移动 Movement
预取 · 迁移
PD 传输 Prefill→Decode

扩展点：可插拔检索与加载策略

▼ Unified KV Storage Interface · 统一 KV 存储接口

③ 全局 KV 存储 Global KV Store

可插拔存储后端

File-System
本地 / 分布式文件系统

KV Semantic · KV Native
语义化 KV 存储

...
更多后端

可靠性 · 保留 · 回收 —— 由各后端能力决定

UCM 是什么

统一缓存管理器：位于**推理引擎与存储之间**的统一 KV Cache 层，统一管理缓存的**复用 · 放置 · 迁移**。

解决什么问题

- 多轮对话 / Agent / 多模态应用，反复访问相同上下文
- KV Cache 困在单个推理进程内：显存有限，重复计算浪费算力

→ **UCM 让 KV Cache 跨进程复用、以外部存储扩容：减少重复计算，缓解显存压力**

三层架构（对应左图）

- **应用与推理引擎** —— 产生并消费 KV Cache
- **统一缓存** —— 复用什么、保留在哪、何时迁移
- **全局 KV 存储** —— 文件系统、内存池等后端的统一抽象

核心设计

引擎适配器、缓存机制、存储后端**可插拔解耦，独立演进**。

Global KV Store：共享、可扩容的 KV存储抽象

把 KV Cache 从「进程私有」变成「全局共享、按需扩容」的资源

推理实例 A | 推理实例 B | Prefill 实例 | Decode 实例

▼ 多方并发读写 · 命中即复用

统一 Store 抽象接口

Lookup · Prefix-Lookup · Prefetch · Load / Dump　按 Block 寻址 · 异步 Task · 后端无关

▼ 写入即入池 · 全局可寻址

弹性 KV 缓存池

容量 = 显存 + 外部存储

显存 HBM + 主机内存 + NVMe · SSD

分布式存储 + … 持续接入

外部存储加入即扩容 —— 容量上限不再由显存决定

空间分配 · 冷热调度 · 层间搬运均由 Store 内部完成，对上层透明

Store 是什么

统一 KV Cache 存储抽象：以 **Block** 为单位寻址，统一提供查找 / 预取 / 装入 / 转储接口——「数据放哪、怎么放」对上层透明。

共享属性 Shared

- KV Cache 不再绑定单个推理进程：写入 Store 即全局可寻址
- 多请求 / 多实例 / 多引擎命中同一份缓存 —— 一份计算、处处复用
- PD 分离：Prefill 与 Decode 实例经 Store 传递 KV

容量可扩展 Scalable

- 缓存容量从「显存上限」变为「显存 + 外部存储」的加法
- 内存、NVMe、分布式文件系统皆可入池：接入即扩容、按需增减
- 空间分配、冷热调度、层间搬运由 Store 内部完成，对上层透明

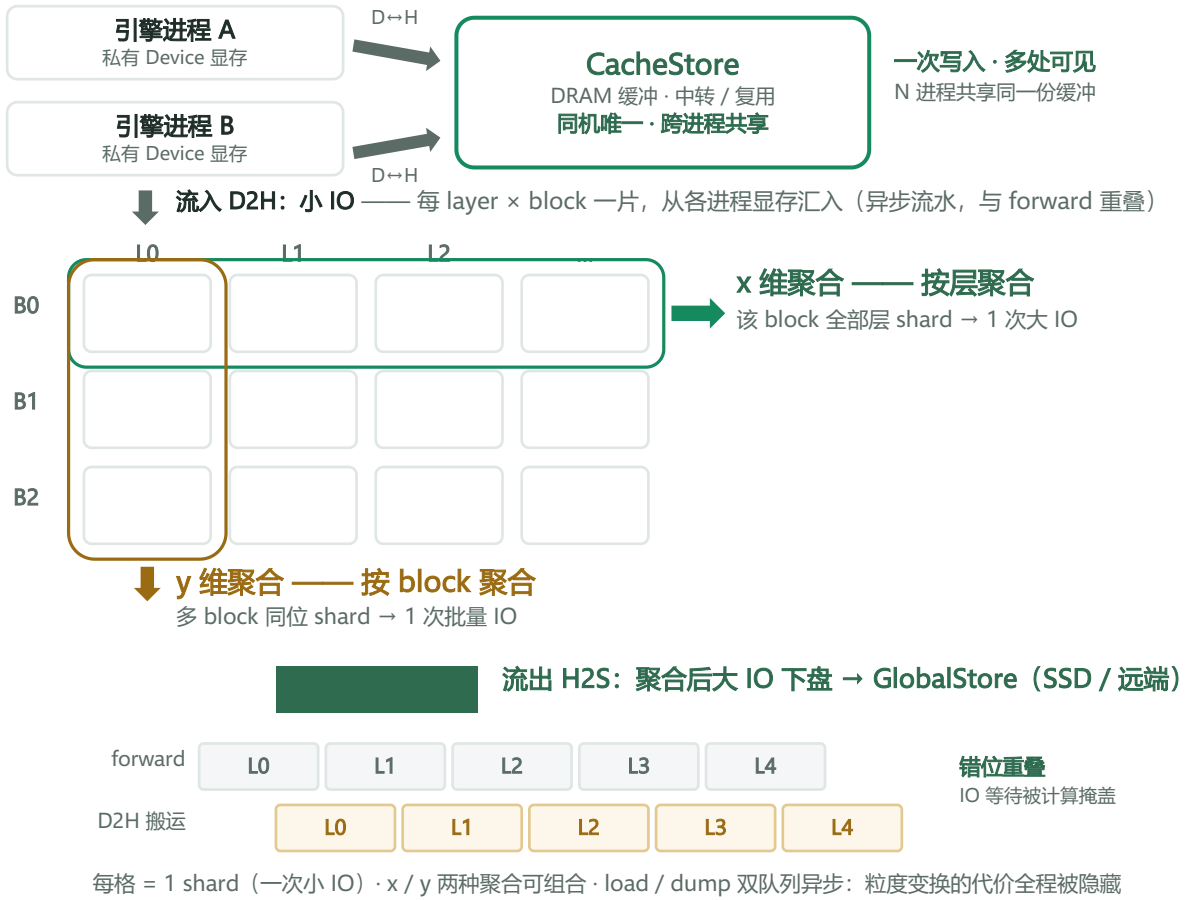
后端各司其职

保留 / 回收 / 可靠性由后端按能力实现；量化、压缩可在传输管线中处理。

扩展：纯 Python / 混合 / 纯 C++　·　接口 ucm/store/ucmstore_v1

CacheStore：节点内轻量共享 Store

同一节点，一次写入 · 多处可见 —— DRAM 共享 × 小 IO 聚合



定位：两域模型的节点内域

CacheStore = 节点内缓存域（HBM/DRAM 中转与复用）；
GlobalStore = 节点外共享域。V1 管线 **Cache|Posix** 的 Cache 即它。

跨进程共享（DRAM 介质）

- **进程内**：多 rank / 多 connector 引用同一 unique_id 缓冲
- **节点内**：/dev/shm 共享缓冲（share_buffer），同机多实例互见
- 效果：一次 dump，同机多处命中 —— 免去重复搬运与重复驻留

小 IO 聚合（io_aggregation）

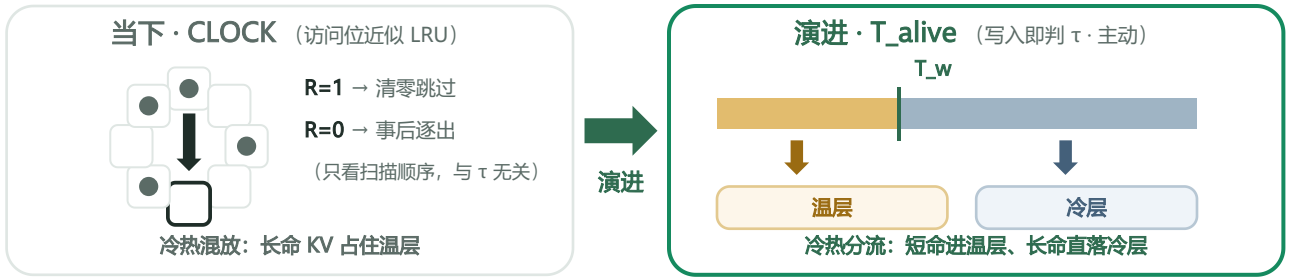
- KV 天然碎片化：每 layer × 每 block 一片，单次 IO 小
- DRAM 缓冲收拢碎片：N 次小 IO → 少量大 IO，摊薄固定开销
- 收益：D↔H 搬运逼近带宽上限，延迟与 CPU 开销同步下降

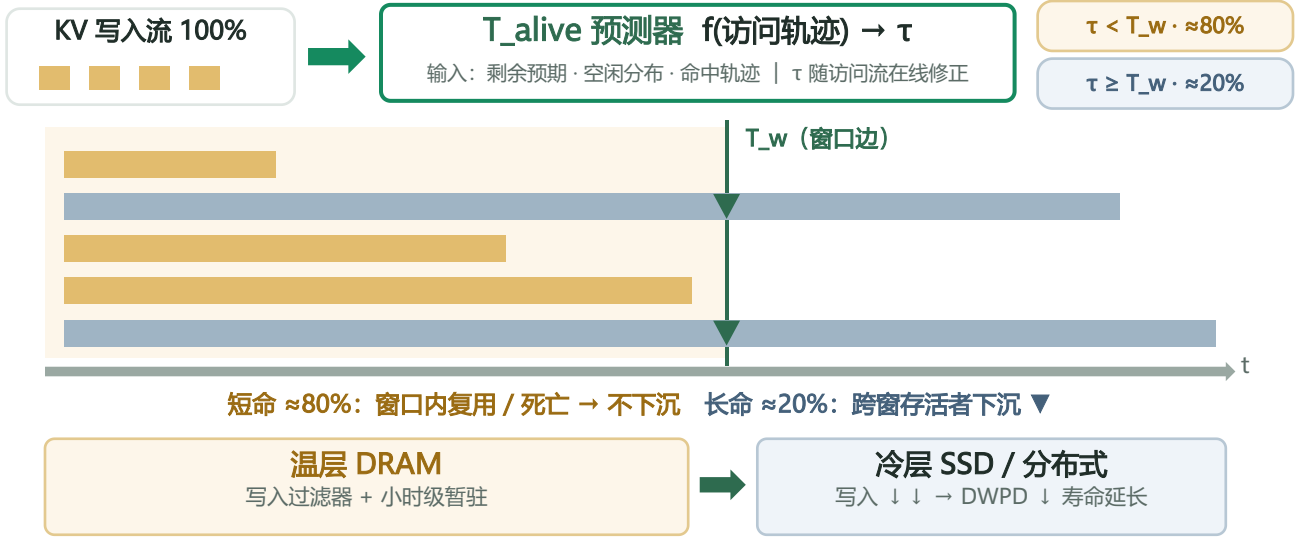
两个聚合维度

- **x · 按层聚合**：整 block 全层 shard → 1 次大 IO（可逐层流水）
- **y · 按 block 聚合**：多 block 同位 shard → 1 次批量 IO
- 维度可组合：一次 task 同时覆盖 blocks × shards

T_alive：预测驱动的分层放置算法

按存活时长分层放置 —— 总容量不变，冷层 DWPD 显著下降





旁路：预测「长命 · 即将回归」→ 回归前预取回热层，命中不落空

DWPD = Drive Writes Per Day（SSD 耐久度指标：每日全盘写入次数）

数据锚点：active ratio @1h ≈ 21%/18%（per-provider，估算）· 中位寿命 57s · 空闲 p50 86s / p99 11.2h（TraceLab 实测）

从 CLOCK 到 T_alive

- **现状**：CLOCK 事后逐出，冷热混放，写洪峰穿透温层
- **演进**：T_alive 写入即判 τ，短命进温层、长命直落冷层
- 同一套缓存设施，决策点从「逐出」前移到「放置」

为什么 T_alive 可预测

- **死亡时间结构性**：多数 KV 死于会话结束（中位 57s），写入即判短命
- **回归时间可统计**：空闲重尾而规律，86.1% 失效紧随 >5min 空闲
- **复用轨迹可推演**：前缀 ≈ 上一步 + 确定性追加（中位误差 2 token）

温层 DRAM = 写入过滤器

- 写入 1h 后仍活跃仅 ~20%——绝大部分写入在温层内消化，从不下沉
- 冷层只接收「预测长命」的存活者：写入流量 100% → ≈20%
- 无温层时冷层直承全部写入 → DWPD 居高不下，被迫选贵介质

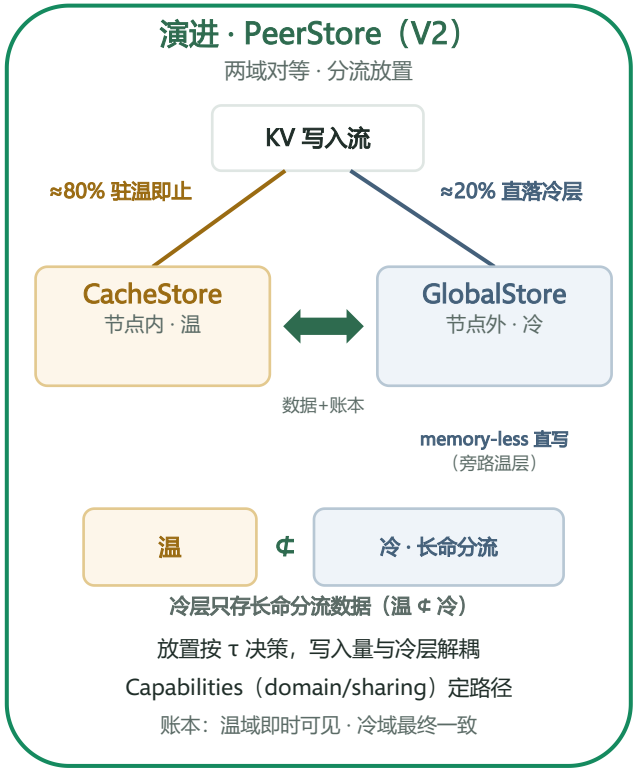
收益闭环

- **总容量不变**：容量仍由冷层承担，温层只是小时级暂驻窗口 + 过滤器
- DWPD 随写入流量同比例下降 → 寿命延长 / 换廉价 QLC → p_store ↓
- **预测越准 → 下沉越少 → DWPD 越低**：预测精度直接兑换成介质成本

Store V2：PeerStore：从「分级缓存」到「分流放置」

存储形态跟着分流走 —— 冷层不再是温层的全量拷贝，该下沉的才下沉





PipelineStore：严格分级缓存（V1）

- Stack 把多级 Store 串成一条传输链（Cache|Posix 等）
- 写路径全量穿层：Device → 温层缓冲 → 冷层逐级直通
- 冷层沉淀全部已下盘数据：**温 ⊆ 冷**

串行链的代价

- 长度阈值准入，不区分死活：短命 KV 也被写穿落冷
- persist_token_threshold 只看长度，不看 τ
- 冷层 DWPD 与写入洪峰绑定：空间 / 寿命双浪费

PeerStore：两域对等放置（V2）

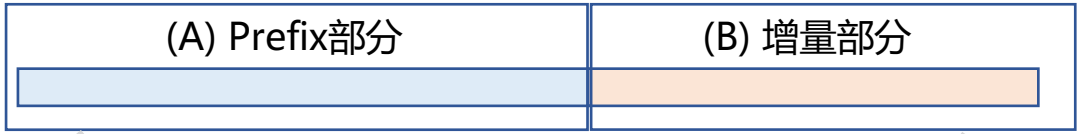
- CacheStore（节点内）// GlobalStore（节点外）平级
- 按 τ 放置：短命驻温不下沉、长命直落冷层（memory-less 直写）
- 冷层只存该存的：**温 ⊄ 冷**，写入量与冷层解耦

支撑机制

- Capabilities（domain/sharing）定放置路径
- **账本双语义**：CacheStore 即时 · GlobalStore 最终一致
- 增量回填仅从缺失层回源 · D↔H 独立直通引擎

编辑与拼接引擎：灵活自由的KV复用范式

推理侧视角：*上下文组成*

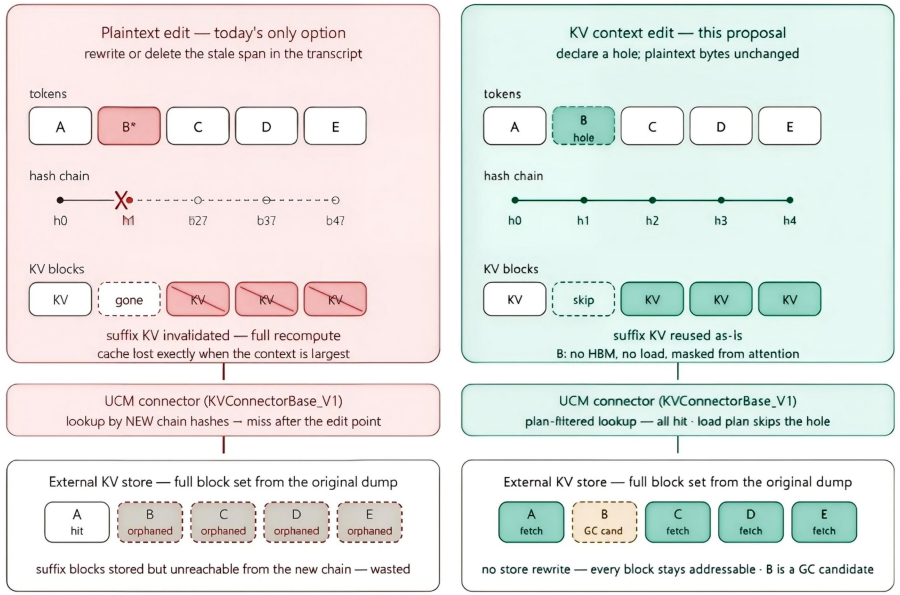


推理代价 $O(A \times B)$

KV编辑

KVBridge编辑引擎：以KV后缀复用代替Agent上下文编辑，避免上下文编辑带来的Prefix命中损失，为Agent提供KV复用接口。

复用范式对比(e.g. 某段Tool Result失效 上下文编辑 vs KVbridge编辑)

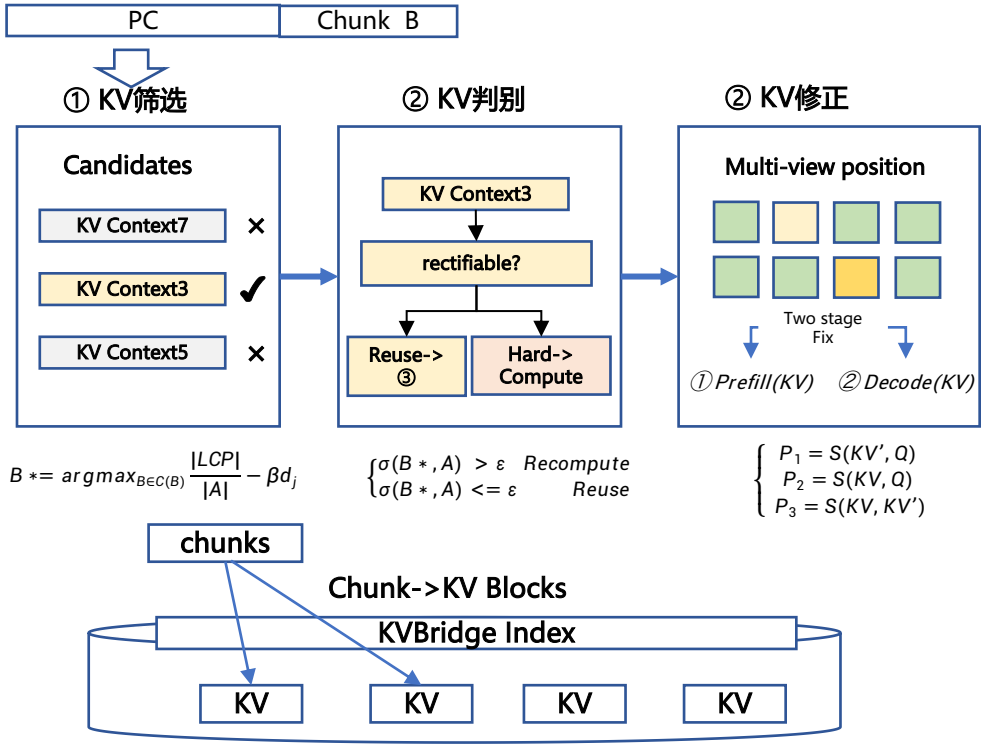


每轮节省推理开销 $\Delta C = \sum_{i \in S} C_p(i)$ $S = \{i | \text{修改位置} e < i < \text{新增位置} n\}$

vllm RFC #58537（欢迎加入共建！）

KV拼接复用

KVBridge拼接引擎：提出①KV筛选->②KV判别->③KV修正全链路离线/在线KV复用，保障推理精度下实现跨位置拼接。



欢迎共建 KV Cache基础设施



GOSIM

GOSIM Shenzhen 2026

Thanks!

